Math4AI Final Capstone Project

National AI Center — AI Academy

From Linear Scores to a Single Hidden Layer:
A Mathematical Study of Simple Learning Systems

Deadline: March 30 at 11:59 PM
Default team size: 3 students
Teams of 4: allowed only upon approved request with clear justification
Required tools: Python, NumPy, Matplotlib, GitHub
Not allowed for core models: PyTorch, TensorFlow, JAX, autograd, scikit-learn model classes

1. Purpose and Framing

This capstone is a **Math4AI-to-AI transition project**. It is **not** a test of prior machine-learning coursework. You are not expected to arrive already knowing standard ML workflow language, neural-network implementation patterns, or model-evaluation conventions. The purpose of this project is to use the mathematics you have already studied to enter a small but central part of AI in a rigorous, guided way.

You are **not** expected to already know neural-network design, professional hyperparameter tuning, advanced ML terminology, or modern uncertainty estimation before starting this project. Part of the capstone is learning these ideas carefully from the background provided here and from the curated references at the end.

The capstone asks one question:

> *When does a one-hidden-layer nonlinear classifier genuinely improve on a linear decision rule, and when is additional model complexity unnecessary?*

That is a mathematically serious question. It connects geometry, chain rule, optimization, likelihood, statistical variability, and scientific judgment. It also has an intellectually honest answer: **more complicated models are not automatically better**. You are **not** expected to outperform the linear baseline everywhere. Part of the project is learning to recognize when a simple model is already sufficient.

2. Conceptual Roadmap

Math4AI foundation	Role in this capstone
Linear Algebra	Affine score functions, vectorized computation, hidden representations, matrix parameters, and optional PCA/SVD for teams choosing Track A.
Calculus and Optimization	Gradients, chain rule, backpropagation, loss minimization, optimizer choice, and training dynamics.
Probability and Statistics	Softmax probabilities, cross-entropy as negative log-likelihood, repeated-seed summaries, and optional reliability analysis for teams choosing Track B.

2.1 What you already know, what is new, and what is optional

You already know	This capstone introduces	You may read further if interested
Vectors, matrices, dot products, least squares, gradients, Hessians, optimization methods, likelihood, MLE, confidence intervals	Supervised learning, classification, train/validation/test protocol, logits, softmax, cross-entropy, baseline models, backpropagation as a computational procedure, overfitting, generalization, reliability analysis	Universal approximation results, richer activation families, advanced optimization methods for deep learning, calibration theory, modern uncertainty estimation

2.2 Compact notation guide

Symbol	Meaning
x	input feature vector for one example
y	true class label for one example
W, b	weight matrices and bias vectors
Z_1	hidden-layer pre-activations for a batch
h or H	hidden representation for one example or a batch
s or S	output scores / logits for one example or a batch
P	matrix of predicted class probabilities
Y	one-hot label matrix for a batch
n	batch size or number of examples under discussion
d	input dimension
k	number of classes

3. Background

3.1 Supervised learning, labels, and classification

In supervised learning, we observe input-output pairs $(x^{(i)}, y^{(i)})$. Here, $x^{(i)}$ is the feature vector and $y^{(i)}$ is the label we want to predict. In this capstone the task is **classification**: each input belongs to one of finitely many classes.

For the two synthetic datasets, the classes are binary. For the digits benchmark, the classes are the ten digits 0 through 9. A classifier takes an input vector and returns either:

- a predicted class label, or
- a vector of class scores or probabilities from which a label is chosen.

3.2 Train, validation, and test data

You will work with a fixed split of the data:

- the **training set** is used to fit parameters,
- the **validation set** is used to choose settings fairly,
- the **test set** is used only for final reporting.

This separation matters. If you choose model settings based on test performance, the test set stops being an honest measure of generalization.

3.3 How the learning pipeline works

At a high level, the workflow is:

1. start with inputs and labels,
2. compute scores or logits from the current parameters,
3. convert scores to class probabilities,
4. compute a loss measuring how well the model matched the labels,
5. compute gradients of that loss with respect to the parameters,
6. update the parameters,
7. repeat this process over training,
8. choose settings on validation data,
9. report final results on the test set.

This capstone asks you to understand each step mathematically, not just to run it procedurally.

3.4 Baseline models

A **baseline model** is a simple reference point used to evaluate whether additional complexity is justified. In this project, softmax regression is the baseline. The neural network is not automatically superior; it must earn its complexity through evidence.

3.5 Why softmax regression is a probabilistic model

The linear model in this project is **softmax regression**. For an input $x \in \mathbb{R}^d$, the model computes class scores

$$s(x) = Wx + b,$$

where $W \in \mathbb{R}^{k \times d}$ and $b \in \mathbb{R}^k$ for k classes. These raw scores are often called **logits**. They are not yet probabilities.

The softmax map turns scores into probabilities:

$$p_j(x) = \frac{\exp(s_j(x))}{\sum_{\ell=1}^{k} \exp(s_\ell(x))}.$$

This ensures that each $p_j(x)$ is nonnegative and that the probabilities sum to one.

This is not merely a convenient formula. It defines a conditional probability model

$$\mathbb{P}(Y = j \mid X = x; W, b) = p_j(x).$$

In other words, the model treats the class label as a random variable whose distribution depends on the input through the logits. The entries of $Wx + b$ therefore play the role of *unnormalized log-probability scores*. Softmax is the normalization map that converts those scores into a valid categorical distribution [2, 4].

3.6 Tiny worked example

Suppose a three-class model produces the score vector

$$s = \begin{bmatrix} 1.2 & 0.2 & -0.4 \end{bmatrix}.$$

Then

$$\exp(s) \approx \begin{bmatrix} 3.32 & 1.22 & 0.67 \end{bmatrix},$$

so the softmax probabilities are approximately

$$p \approx \begin{bmatrix} 0.64 & 0.23 & 0.13 \end{bmatrix}.$$

If the true class is the first class, then the cross-entropy loss is approximately

$$-\log(0.64) \approx 0.45.$$

If the true class were instead the second class, the loss would be

$$-\log(0.23) \approx 1.47.$$

The mechanism is simple: the model is rewarded when it puts high probability on the correct label and penalized when it does not.

3.7 Why cross-entropy is negative log-likelihood

If the true class for an example is y, the cross-entropy loss is

$$\mathcal{L}(x, y) = -\log p_y(x).$$

Intuitively, this loss punishes the model when it assigns small probability to the correct class. Probabilistically, if the observed label is treated as a draw from the model's categorical distribution, then the likelihood of observing the correct class is exactly $p_y(x)$, and the log-likelihood is $\log p_y(x)$. Negating that quantity gives the loss above. Therefore, minimizing average cross-entropy over the dataset is the same as maximizing the average log-likelihood of the observed labels under the model [2, 4]. This matters because it connects classification directly to probabilistic modeling rather than to an arbitrary penalty function.

3.8 Linear score functions and affine decision boundaries

A multiclass linear classifier computes

$$s(x) = Wx + b.$$

The decision boundary between classes j and ℓ is determined by the equation

$$s_j(x) = s_\ell(x),$$

which is affine in x. In two dimensions, such boundaries are lines; in higher dimensions, they are hyperplanes.

3.9 One-hidden-layer network

The neural network in this capstone has one hidden layer:

$$h = \tanh(W_1 x + b_1), \qquad s = W_2 h + b_2.$$

The output scores s are then passed through softmax exactly as in the linear model. This model composes an affine map, a nonlinear activation, and a second affine map [3, 5, 7].

3.10 Why `tanh` was chosen for this capstone

Modern practice often favors ReLU-like activations, but this capstone uses `tanh` for pedagogical reasons. First, `tanh` is smooth and its derivative has a compact closed form:

$$\frac{d}{dz}\tanh(z) = 1 - \tanh^2(z).$$

That makes the backpropagation derivation cleaner. Second, `tanh` is zero-centered, which simplifies interpretation of hidden activations and often yields less awkward update behavior than the sigmoid family. Third, because this project is about understanding representational change rather than maximizing performance on a large benchmark, the clarity of the mathematics matters more than imitating current large-scale engineering defaults. You may mention ReLU in your report if useful, but the required core model uses `tanh`.

3.11 Backpropagation before equations

Backpropagation is not magic and it is not a separate learning principle. It is a bookkeeping procedure for applying the chain rule efficiently through a composed computation.

In the forward pass, you compute intermediate quantities: affine pre-activations, hidden activations, output scores, probabilities, and loss. In the backward pass, you propagate sensitivities of the loss with respect to those quantities in reverse order. Each local derivative is simple; the chain rule assembles them into gradients with respect to the parameters [6, 4, 3].

Conceptually, backpropagation answers the question:

> *If the loss changed slightly, which parameters are responsible, in which direction, and by how much?*

3.12 Backpropagation in vectorized form

For implementation, you should think in batches. If a batch of inputs is stored row-wise in a matrix $X \in \mathbb{R}^{n \times d}$, then a vectorized forward pass can be written as

$$Z_1 = XW_1^\top + \mathbf{1}b_1^\top, \qquad H = \tanh(Z_1),$$
$$S = HW_2^\top + \mathbf{1}b_2^\top, \qquad P = \mathrm{softmax}_{\mathrm{row}}(S),$$

where $P \in \mathbb{R}^{n \times k}$ contains one predicted probability vector per row. If Y is the one-hot label matrix, then the average batch cross-entropy leads to the familiar output-layer sensitivity

$$\frac{\partial \mathcal{L}}{\partial S} = \frac{1}{n}(P - Y).$$

From there, the chain rule propagates backward:

$$\frac{\partial \mathcal{L}}{\partial W_2} = \left(\frac{\partial \mathcal{L}}{\partial S}\right)^\top H, \qquad \frac{\partial \mathcal{L}}{\partial b_2} = \left(\frac{\partial \mathcal{L}}{\partial S}\right)^\top \mathbf{1},$$
$$\frac{\partial \mathcal{L}}{\partial Z_1} = \left(\frac{\partial \mathcal{L}}{\partial S}\right) W_2 \odot (1 - H \odot H),$$
$$\frac{\partial \mathcal{L}}{\partial W_1} = \left(\frac{\partial \mathcal{L}}{\partial Z_1}\right)^\top X, \qquad \frac{\partial \mathcal{L}}{\partial b_1} = \left(\frac{\partial \mathcal{L}}{\partial Z_1}\right)^\top \mathbf{1}.$$

These formulas are not an optional implementation trick. They are the matrix form of the same chain rule you know from calculus. Vectorization simply organizes many scalar derivatives into a compact linear-algebra computation.

3.13 Overfitting, generalization, and fair comparison

Overfitting occurs when a model fits the training data very closely but does not generalize well to new data. **Generalization** is the ability to perform well on unseen examples from the same task distribution.

Because the two model classes differ in structure and parameterization, comparison must be fair. Fair comparison means:

- same train/validation/test split,
- same preprocessing rules,
- same reporting metrics,
- model settings chosen using validation performance rather than test performance,
- repeated-seed reporting on the digits benchmark for the final selected configurations.

3.14 Repeated-seed evaluation

Optimization can depend on random initialization and mini-batch order, so repeated runs of the same training procedure may not produce identical results.

In this project you will use five random seeds on the digits benchmark for the final selected configurations and summarize the outcomes with means and 95% confidence intervals.

3.15 Recommended references

Use these selectively. They are references, not a second syllabus.

- Deisenroth, Faisal, and Ong, *Mathematics for Machine Learning* [1]
- Murphy, *Probabilistic Machine Learning: An Introduction* [2]
- Goodfellow, Bengio, and Courville, *Deep Learning* [3]
- Ng and Katanforoosh, CS229 deep learning notes [4]
- Stanford CS231n notes on neural networks and backpropagation [5, 6]
- ETH Introduction to Machine Learning notes [7]
- GitHub Flow documentation and MIT Missing Semester notes for version control [8, 9]

4. Data and Starter Pack

4.1 Common data for all teams

All teams must use the same data and splits.

1. **Linear synthetic task:** two Gaussian blobs with mild overlap.
2. **Nonlinear synthetic task:** moons.
3. **Digits benchmark:** fixed digits dataset with shared train/validation/test split.

The synthetic tasks are chosen for geometric clarity. The digits benchmark is chosen because it is small, standard, clean, and substantial enough to make the comparison meaningful without turning the project into a data-engineering exercise.

Use the starter-pack files directly:

- `starter_pack/data/linear_gaussian.npz` for the linear synthetic task
- `starter_pack/data/moons.npz` for the nonlinear synthetic task
- `starter_pack/data/digits_data.npz` for the digits feature matrix and labels
- `starter_pack/data/digits_split_indices.npz` for the fixed train/validation/test indices

4.2 Exact digits benchmark protocol

The digits benchmark protocol is fixed for all teams.

- **Input representation:** use the flattened 64-dimensional vectors stored in `starter_pack/data/digits_data.npz`.
- **Labels:** use the digit labels stored in `starter_pack/data/digits_data.npz`.
- **Split definition:** use `train_idx`, `val_idx`, and `test_idx` from `starter_pack/data/digits_split_indices.npz`. The script `starter_pack/scripts/make_digits_split.py` is included for reproducibility; your experiments should use the published indices.
- **Scaling:** the provided digits features are already scaled to the interval $[0, 1]$. Do not apply additional normalization, whitening, feature engineering, or augmentation in the required core experiments.
- **Core preprocessing rule:** outside Track A, use the digits features exactly as provided in `starter_pack/data/digits_data.npz`.
- **Evaluation metrics:** for digits, always report both classification accuracy and mean cross-entropy loss. Accuracy means the fraction of correctly classified examples. Mean cross-entropy means the average negative log-probability assigned to the true class over the evaluation split.
- **Model selection metric:** use validation cross-entropy as the primary metric when choosing settings.
- **Default hidden width:** use hidden width 32 for the neural network outside the required capacity ablation.
- **Default regularization:** use L_2 regularization with coefficient $\lambda = 10^{-4}$ for both models.
- **Default batch size:** use mini-batch size 64.
- **Softmax baseline optimizer:** use mini-batch SGD with learning rate 0.05.
- **Optimizer defaults for the required neural-network optimizer study:** use SGD with learning rate 0.05, Momentum with learning rate 0.05 and momentum coefficient 0.9, and Adam with learning rate 0.001, $\beta_1 = 0.9$, $\beta_2 = 0.999$, and $\varepsilon = 10^{-8}$.
- **Epoch budget:** train for at most 200 epochs and use the best validation-cross-entropy checkpoint from within that budget.

These defaults are part of the experimental contract. Do not turn the digits benchmark into an open-ended hyperparameter search.

4.3 Starter pack contents

The starter pack is intentionally minimal. It includes only:

- `starter_pack/data/digits_data.npz`,
- `starter_pack/data/digits_split_indices.npz`,
- `starter_pack/data/linear_gaussian.npz`,
- `starter_pack/data/moons.npz`,
- `starter_pack/scripts/make_digits_split.py`,
- `starter_pack/scripts/generate_synthetic.py`,
- `starter_pack/README.md` and `starter_pack/CHECKLIST.md`,
- the optional report template `starter_pack/report/template.tex`.

It does **not** include any model implementation code, model skeletons, placeholder methods, optimizer scaffolding, or training-loop scaffolding.

Begin with `starter_pack/README.md` and `starter_pack/CHECKLIST.md`. If you want to inspect how the provided datasets were produced, read `starter_pack/scripts/generate_synthetic.py` and `starter_pack/scripts/make_digits_split.py`.

4.4 How to access the repository

The student repository for this capstone is:

https://github.com/Murad-Huseynli/math4ai_capstone

Clone it with:

```
git clone https://github.com/Murad-Huseynli/math4ai_capstone.git
```

After cloning, begin by reading `README.md`, then `starter_pack/README.md`, and then `starter_pack/CHECKLIST.md`.

5. Implementation Requirements

5.1 What "from scratch" means

You must implement the intellectually central parts of the project yourself in NumPy. In particular, you must implement:

- numerically stable logits and softmax probabilities,
- cross-entropy loss,

- forward propagation,
- backpropagation for both models,
- parameter updates,
- mini-batch training,
- evaluation metrics,
- the core plots used in your analysis.

You may use NumPy's linear algebra routines. Teams choosing Track A may use `np.linalg.svd`. You are not expected to reimplement SVD.

5.2 Allowed and prohibited tools

Allowed	Python, NumPy, Matplotlib, standard-library utilities, light helper scripts, LaTeX or another professional PDF workflow
Not allowed for the core methods	PyTorch, TensorFlow, JAX, autograd, scikit-learn model classes, hidden high-level training loops, external datasets, leaderboard-style competition framing

5.3 AI-tool usage policy

You may use AI tools for limited support tasks such as debugging, explaining a concept, checking syntax, or clarifying an error message. However, AI tools do **not** replace understanding. You remain fully responsible for:

- correctness,
- conceptual understanding,
- implementation integrity,
- oral defense of every major idea and code path you submit.

If you cannot explain or defend a major claim or implementation decision during the presentation, then it does not count as understood work.

5.4 Required models

Model 1: multiclass softmax regression.
Model 2: one-hidden-layer neural network with `tanh` hidden activations and softmax output.

Use a single hidden layer only. Do not turn this into a deeper-network project.

5.5 Implementation sanity checks

Your report must contain a short subsection titled **Implementation Sanity Checks** or **Verification and Debugging Checks**. This is a required part of the project.

Include a concise record of at least **three** checks such as:

- a gradient sanity check on a tiny batch or selected parameter entries,
- evidence that the loss decreases on a tiny subset after a few updates,
- successful overfitting of a very small subset of training examples,
- confirmation that predicted class probabilities sum to one,
- confirmation that training did not silently produce NaNs or infinities.

Keep this subsection short and concrete. Its purpose is to show implementation discipline, not to become a second report.

6. Required Mathematical Work

Your report must at least include the following (the deeper your analysis beyond these points, the better for you):

1. A derivation showing why softmax cross-entropy can be interpreted as negative log-likelihood.
2. A conceptual explanation of backpropagation before giving equations.
3. A complete derivation of the parameter gradients for the one-hidden-layer network.
4. A mathematical explanation of why the Gaussian task is well matched to a linear decision rule and why the moons task is not.

You may discuss regularization as an interpretive idea, but do not inflate that discussion into a second theory project.

7. Experimental Program

Depth of interpretation matters more than the number of experiments. A small number of well-designed experiments is better than a long list of shallow runs. Raw accuracy is not the objective. Mathematical and scientific understanding is.

7.1 Required core experiments

Every team must complete the following comparisons:

1. Train and compare both models on the linear Gaussian task. Include decision-boundary plots.

2. Train and compare both models on the moons task. Include decision-boundary plots.
3. Train and compare both models on the fixed digits benchmark using the same preprocessing and split.

7.2 Checkpoint policy across datasets

Use a simple and consistent checkpoint policy:

- For the two synthetic tasks, you may report the final epoch within your stated training budget unless you explicitly justify a different choice.
- For the digits benchmark, you must use the best validation-cross-entropy checkpoint within the allowed epoch budget.

7.3 Required ablations

You must complete exactly these three:

1. **Capacity ablation on moons.** Use hidden widths $\{2, 8, 32\}$. Interpret what changes in the learned decision boundary.
2. **Optimizer study on digits.** On the one-hidden-layer neural network with hidden width 32, compare exactly these optimizers: SGD, Momentum, and Adam, using the default settings stated in the digits protocol. Softmax regression is not part of this optimizer comparison; it remains the fixed baseline model. Use the same split, preprocessing, epoch budget, metrics, and checkpoint rule for all three optimizers. The goal is to compare training behavior under fixed conditions, not to perform separate hyperparameter tuning for each optimizer.
3. **One failure-case analysis.** This may come from under-capacity, an optimizer behaving poorly, instability, or visible overfitting. You must explain *why* it failed, not merely show that it failed.

7.4 Repeated-seed statistics

For the digits benchmark, model selection and final reporting must follow this exact protocol:

1. Use the training set to fit parameters.
2. Use the validation set only to choose settings and select the best epoch.
3. Fix one final selected configuration for softmax regression and one for the neural network using validation evidence only.
4. For each selected configuration, run **five seeds**.
5. For each seed, evaluate the test set exactly once after the configuration and best epoch have been selected.

You must then report:

- mean test accuracy over 5 seeds,
- mean test cross-entropy over 5 seeds,
- a 95% confidence interval for each mean,
- a short interpretation of what the variability does and does not justify.

With five seeds, an acceptable 95% confidence interval for the sample mean is

$$\bar{x} \pm 2.776 \cdot \frac{s}{\sqrt{5}},$$

where $\bar{x}$ is the sample mean and s is the sample standard deviation. If you use a different correct method, state it clearly.

7.5 Visualizations

At minimum, your report must contain:

- decision-boundary plots for the two synthetic tasks,
- at least one training-dynamics figure for the digits benchmark,
- a figure or table supporting your advanced track,
- a figure or table dedicated to the failure case.

8. Advanced Analysis Track

Every team must complete exactly **one** of the following tracks.

8.1 Track A: PCA/SVD and input geometry

This is a **focused analysis**, not a second large experiment grid. Its purpose is to study input geometry and dimensional compression before classification, and to compare linear structure in the input to what nonlinear representation learning may add.

Required work:

- one scree plot,
- one 2D PCA visualization of the digits data,
- one small softmax comparison at fixed PCA dimensions $m \in \{10, 20, 40\}$,
- a brief interpretation tied back to the main project question.

Use the digits data in `starter_pack/data/digits_data.npz` for this track.

8.2 Track B: Prediction confidence and reliability

Track B must be conducted on the fixed digits benchmark. Use predictions produced by your trained models on the fixed digits benchmark, using inputs from `starter_pack/data/digits_data.npz` and the split defined in `starter_pack/data/digits_split_indices.npz`. This is **not** a full modern uncertainty-estimation project. Keep the language precise and modest: confidence, reliability, confidence versus correctness, predictive entropy.

Required work:

- confidence defined as the maximum predicted class probability,
- predictive entropy from the softmax output,
- a 5-bin confidence-versus-empirical-accuracy table or plot for both models,
- a comparison of confidence or entropy for correct versus incorrect predictions,
- a brief interpretation tied back to the main project question.

9. Deliverables

9.1 Repository

You must submit a GitHub repository link. The repository must include:

- meaningful commit history,
- branch-based workflow,
- at least one merged feature branch per team member,
- readable organization,
- exact reproduction instructions in the README.

9.2 Final report

Submit a professionally formatted PDF report.

- Length: 6–8 pages of main text
- References do not count toward the page limit
- An appendix is optional
- LaTeX/Overleaf is strongly recommended but not required

An optional starting template is included at `starter_pack/report/template.tex`.

Your report must include the following sections:

- Introduction / framing

- Background and mathematical setup
- Methods
- Implementation sanity checks
- Experiments
- Advanced track
- Discussion
- **Limitations**
- References

The **Limitations** subsection is required. It should explain what your study does not test, what claims remain uncertain, and why your conclusions should not be overgeneralized.

Your report must do more than display equations and plots. It must explain:

- what each model class can represent geometrically,
- what evidence supports each major conclusion,
- where the linear model is sufficient,
- where the hidden layer changes the geometry of the problem,
- what the limits of those conclusions are.

Your report must also explicitly answer the following interpretive questions. These are questions for *your team* to answer from mathematics and evidence; do not treat them as conclusions supplied by the handout:

- When is the linear model geometrically sufficient, and when is it not?
- What representational change does the hidden layer provide in your experiments?
- When does additional model complexity fail to justify itself?
- What does your failure case teach about optimization, capacity, or generalization?
- What do repeated-seed statistics add beyond a single run?

Strong reports argue from evidence rather than from preference. If the linear baseline is sufficient for part of the project, say so clearly and justify that claim. **The deeper your analysis and beyond the points you were asked, the better for you.**

Use consistent notation throughout the report and define symbols before using them.

9.3 Slides and presentation

You must deliver a **10-minute technical presentation** followed by **5 minutes of committee Q&A**. Every team member must speak.

Treat this as a technical pitch to knowledgeable reviewers. Your presentation should emphasize:

- the central question,
- why the comparison is fair,
- where linear structure was sufficient,
- where nonlinearity helped,
- what the failure case reveals,
- what repeated-seed evaluation added beyond a single run,
- what limitations remain.

Your presentation must explicitly answer, in concise form, the same core interpretive questions required in the report. These are questions for your team to answer from its own evidence: when the linear model is geometrically sufficient, what representational change the hidden layer provides, when extra complexity does not justify itself, what the failure case teaches, and what repeated-seed statistics add.

9.4 Contribution statement

Each team member must submit a short statement describing their contributions. This is primarily for accountability and conflict resolution, not for performative individualization.

10. GitHub and Collaboration Expectations

GitHub is required because this capstone is collaborative, technical, and cumulative. However, Git is not the point of the project. Use it as professional infrastructure, not as a substitute for scientific thinking.

10.1 Minimum collaboration standard

- Default team size is 3; teams of 4 are allowed only upon approved request with clear justification.
- Every member must contribute through a branch-based workflow.
- Every member must have at least one merged feature branch.
- Commit messages must be meaningful.
- The README must explain setup, repository structure, and reproduction.

10.2 Compact Git primer

Command	Purpose
`git clone <url>`	Copy the repository to your machine
`git status`	Check what changed and what is staged
`git add <file>`	Stage changes for the next commit
`git commit -m "..."`	Create a snapshot with a meaningful message
`git pull`	Bring in remote changes before continuing work
`git push`	Publish your commits to GitHub
`git branch`	List or create branches
`git switch <name>`	Move to another branch
or	
`git checkout <name>`	
`git merge <name>`	Merge a completed branch into your current branch

10.3 Repository organization

A simple layout is enough. In the distributed repository, the working scaffold lives under `starter_pack/`:

- `starter_pack/`
- `starter_pack/data/`
- `starter_pack/scripts/`
- `starter_pack/src/`
- `starter_pack/figures/`
- `starter_pack/results/`
- `starter_pack/report/`
- `starter_pack/slides/`
- `README.md`

Your README should state:

- the project objective,
- how to set up the environment,
- how to reproduce the main experiments,
- what each major folder contains,
- a high-level summary of team contributions.

11. Recommended One-Week Workflow

This is not graded, but it is realistic.

1. **Day 1:** Read the handout carefully, inspect the data, assign responsibilities, and set up the repository.
2. **Day 2:** Implement and verify softmax regression.
3. **Day 3:** Implement and verify the one-hidden-layer network and backpropagation.
4. **Day 4:** Complete the required core comparisons on the synthetic tasks and digits.
5. **Day 5:** Run the required ablations, optimizer study, and repeated-seed evaluation.
6. **Day 6:** Complete one advanced track, organize figures, draft the report.
7. **Day 7:** Finalize the report, clean the repository, and rehearse the presentation.

12. Evaluation Rubric

Category	What we are looking for	Weight
Mathematical understanding and derivations	Correct derivations of softmax negative log-likelihood and network gradients; clear explanation of what the linear model and the hidden-layer model can represent; accurate use of notation; and mathematically serious reasoning about geometry rather than unsupported intuition	20
Implementation correctness and numerical care	Stable softmax implementation, correct cross-entropy, correct vectorized backpropagation, sensible parameter updates, and visible implementation discipline through sanity checks such as gradient verification, probability checks, tiny-subset behavior, or NaN/Inf checks	16
Experimental design and fair comparison	Same split and preprocessing for both models, validation-only model selection, correct digits protocol, completion of exactly the required capacity ablation and optimizer study, and experiments designed to answer the central question rather than pad the report	18
Interpretation, repeated-seed statistics, advanced-track analysis, and failure analysis	Claims supported by figures or tables, repeated-seed statistics computed and interpreted correctly, explicit answers to the required conceptual questions, advanced track tied back to the main story, and failure analysis that explains a mechanism rather than merely showing a poor result	18
Report clarity, figures, explanation of new AI concepts, and limitations	New AI terms defined clearly, figures readable and discussed, notation used consistently, conclusions linked to evidence, and limitations stated honestly with explicit boundaries on what the study does not show	10
Presentation and oral defense	Clear central claim, fair summary of evidence, balanced participation by team members, and precise, honest responses to conceptual, mathematical, and implementation questions during Q&A	10
Repository quality and collaboration hygiene	Meaningful commit history, readable README, exact reproduction instructions, branch-based workflow, and clear evidence that all members contributed through merged work	8

There is **no standalone rubric category for raw accuracy**. Strong work is defined by correctness, judgment, explanation, and honest evidence.

13. Submission Summary

Submit by March 29 at 11:59 PM:

- GitHub repository link
- final PDF report
- slide deck
- contribution statement from each team member

Prepare for a live 10-minute presentation and 5-minute Q&A with the review committee.

References

References

[1] Marc Peter Deisenroth, A. Aldo Faisal, and Cheng Soon Ong. *Mathematics for Machine Learning*. Cambridge University Press, 2020.

[2] Kevin P. Murphy. *Probabilistic Machine Learning: An Introduction*. MIT Press, 2022.

[3] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, 2016.

[4] Andrew Ng and Kian Katanforoosh. *CS229 Lecture Notes: Deep Learning*. Stanford University, updated notes available at https://cs229.stanford.edu/.

[5] Stanford CS231n staff. *Neural Networks Part 1: Setting up the Architecture*. https://cs231n.github.io/neural-networks-1/

[6] Stanford CS231n staff. *Optimization Part 2: Backpropagation, Intuitions*. https://cs231n.github.io/optimization-2/

[7] ETH Zurich Learning and Adaptive Systems Group. *Introduction to Machine Learning Course Notes*. https://las.inf.ethz.ch/teaching/introml-s24

[8] GitHub. *GitHub Flow*. https://docs.github.com/en/get-started/using-github/github-flow

[9] MIT CSAIL. *The Missing Semester of Your CS Education: Version Control (Git)*. https://missing.csail.mit.edu/2020/version-control/